

Profiling report

Measurement Environment

This profiling report measures the final embedding-based face verification inference pipeline on a CPU baseline. The system was run locally on Windows using the project virtual environment on an Intel(R) Core(TM) i5-8350U CPU @ 1.70GHz. No GPU comparison was included. The input pairs came from outputs/pairs/val.csv, and the final operating threshold was 0.30612244897959173.

The final system uses InceptionResnetV1 pretrained on VGGFace2 from facenet-pytorch for embedding generation. Cosine similarity is used as the scoring method. The profiling script measured preprocessing latency, embedding-generation latency, scoring latency, and total end-to-end latency.

Methodology

The profiling script was run on deterministic validation pairs from the LFW dataset. For each image pair, the script measured preprocessing time, embedding-generation time, scoring time, and total inference time using `time.perf_counter()`.

To evaluate batch-size sensitivity, the script was run with batch sizes of 1, 5, 10, and 25 pairs. For each batch size, the report records average preprocessing latency, average embedding latency, average scoring latency, average end-to-end latency, p95 end-to-end latency, and throughput in pairs per second.

The command used was:

```
python scripts/profiling.py --config configs/m4.yaml --pairs outputs/pairs/val.csv --batch_sizes 1 5 10 25
```

CPU Baseline

Batch Size	Num Pairs	Avg Preprocess (s)	Avg Embedding (s)	Avg Scoring (s)	Avg Total (s)	p95 Total (s)	Throughput (pairs/s)
1	1	.377132	.677846	.014323	1.069527	1.069527	.934993
5	5	.004704	.134905	.000180	.139791	.15578	7.153525

						0	
10	10	.003997	.134187	.000143	.138329	.155898	7.229142
25	25	.003659	.127452	.000129	.131241	.163935	7.619550

Interpretation

The profiling results show that embedding generation is the dominant runtime cost in the final system. For the larger batch-size runs, average embedding time was around 0.127 to 0.135 seconds per pair, while scoring time was close to zero, around 0.0001 to 0.0002 seconds per pair. This makes sense because cosine similarity is a lightweight vector operation, while embedding generation requires forward passes through the pretrained InceptionResnetV1 model.

The batch-size 1 run had a noticeably higher preprocessing time and total latency than the larger runs. This likely reflects startup overhead from loading cached data, model warmup, or one-time TensorFlow/PyTorch initialization effects. After the first run, preprocessing time dropped significantly, and the average end-to-end latency stabilized around 0.13 to 0.14 seconds per pair.

Batch-size sensitivity shows that throughput improved as the number of processed pairs increased. Throughput rose from about 0.93 pairs per second for batch size 1 to about 7.62 pairs per second for batch size 25. The p95 latency values stayed relatively close to the average total latency values, which suggests that the slower requests were not dramatically worse than the typical request after the system warmed up. Together, these metrics give a fuller view of runtime behavior, with average latency showing typical performance, p95 latency capturing slower-end requests, and throughput showing how many image pairs the system can process per second.

These measurements demonstrate a local CPU baseline and should not be treated as production performance guarantees. Runtime may vary depending on CPU model, memory availability, operating system, Docker overhead, dataset cache state, and whether model weights have already been downloaded.